

MyCESA — Application de Gestion Scolaire

Application complète de gestion scolaire avec backend REST API, application mobile React Native et interface web admin.

État du projet

Module	Statut
 Backend API REST	Terminé
 App Mobile (React Native + Expo)	En cours
 Interface Web Admin (React)	À venir

Prérequis — Logiciels à installer

Chaque collaborateur doit installer les logiciels suivants **dans cet ordre** :

1. Node.js v20 LTS

- Télécharger sur : <https://nodejs.org>
- Choisir **LTS (Long Term Support)**
- Vérifier l'installation : `node -v` → doit afficher `v20.x.x`

2. Git

- Télécharger sur : <https://git-scm.com>
- Vérifier : `git -v`

3. WampServer (pour la base de données MySQL)

- Télécharger sur : <https://www.wampserver.com>
- Lancer WampServer → icône doit être **verte**
- Accéder à phpMyAdmin : <http://localhost/phpmyadmin>

4. VS Code (éditeur recommandé)

- Télécharger sur : <https://code.visualstudio.com>
- Extensions recommandées :

- **ES7+ React/Redux/React-Native snippets**
- **Prettier - Code formatter**
- **GitLens**
- **REST Client** (pour tester l'API)

5. Postman (pour tester l'API)

- Télécharger sur : <https://www.postman.com>

6. Expo Go (sur téléphone Android/iOS)

- Android : <https://play.google.com/store/apps/details?id=host.exp.exponent>
- iOS : <https://apps.apple.com/app/expo-go/id982107779>
- ⚠ Version requise : **SDK 54**

Installation du projet

Étape 1 — Cloner le projet

```
bash

git clone https://github.com/bossombra1/MyCESA.git
cd MyCESA
```

Étape 2 — Configurer la base de données

1. Ouvre **phpMyAdmin** → <http://localhost/phpmyadmin>
2. Crée une base de données nommée **mycesa_db** (encodage : **utf8mb4_unicode_ci**)
3. Importe le fichier SQL :
 - Clique sur **mycesa_db** → **Importer**
 - Sélectionne le fichier **backend/database/schema.sql**
 - Clique sur **Exécuter**

Étape 3 — Configurer le Backend

```
bash

cd backend
npm install
```

Crée un fichier **.env** dans le dossier **backend/** :

```
env
```

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=mycesa_db
DB_PORT=3306
JWT_SECRET=mycesa_super_secret_key_2026_CHANGE_MOI
PORT=3000
```

Lance le backend :

```
bash

npm run dev
```

✅ Le backend tourne sur : <http://localhost:3000>

Étape 4 — Configurer l'App Mobile

```
bash

cd ../mobile
npm install --legacy-peer-deps
```

⚠️ **Important** : Trouve ton adresse IP WiFi :

```
bash

ipconfig
```

Cherche **Adresse IPv4** sous **Carte réseau sans fil Wi-Fi** (exemple : `192.168.1.69`)

Ouvre le fichier `mobile/src/api/api.js` et remplace l'IP :

```
javascript

const BASE_URL = 'http://TON_IP:3000/api';
```

Lance l'app mobile :

```
bash
```

```
# PowerShell
```

```
$env:REACT_NATIVE_PACKAGER_HOSTNAME="TON_IP"
```

```
npx expo start -c --lan
```

```
# Si le téléphone est connecté via point d'accès PC
```

```
$env:REACT_NATIVE_PACKAGER_HOSTNAME="192.168.137.1"
```

```
npx expo start -c --lan
```

Scanne le QR code avec **Expo Go** sur ton téléphone.

Lancer le projet chaque jour

Terminal 1 — Backend :

```
bash
```

```
cd backend
```

```
npm run dev
```

Terminal 2 — Mobile :

```
bash
```

```
cd mobile
```

```
$env:REACT_NATIVE_PACKAGER_HOSTNAME="TON_IP"
```

```
npx expo start --lan
```

Structure du projet

```
MyCESA/
```

```
|— backend/          # API REST Node.js + Express
```

```
| |— config/
```

```
| | |— db.js          # Connexion MySQL
```

```
| |— database/
```

```
| | |— schema.sql     # Structure + données initiales
```

```
| |— middleware/
```

```
| | |— authMiddleware.js # Vérification JWT
```

```
| |— routes/          # Toutes les routes API
```

```
| | |— auth.js
```

```
| | |— etudiants.js
```

```
| | |— classes.js
```

```
| | |— evaluations.js
```



API — Routes disponibles

Route	Méthodes	Description
<code>/api/auth</code>	POST /register, POST /login, GET /me	Authentification
<code>/api/etudiants</code>	GET, POST, PUT, DELETE	Gestion étudiants
<code>/api/classes</code>	GET, POST, PUT, DELETE	Gestion classes

Route	Méthodes	Description
<code>/api/evaluations</code>	GET, POST, PUT, DELETE	Notes et évaluations
<code>/api/absences</code>	GET, POST, PUT	Suivi absences
<code>/api/versements</code>	GET, POST	Paiements scolarité
<code>/api/emploiTemps</code>	GET, POST, DELETE	Emploi du temps
<code>/api/chatbot</code>	POST /ask, GET /historique	Assistant IA
<code>/api/notifications</code>	GET, POST, PUT	Notifications
<code>/api/professeurs</code>	GET, POST, PUT, DELETE	Gestion professeurs

Compte de test

Login : admin
Password : admin123
Rôle : Administrateur

Workflow Git (collaboration)

```
bash

# Avant de commencer à travailler
git pull origin main

# Créer une branche pour ta fonctionnalité
git checkout -b feature/nom-de-la-fonctionnalite

# Après avoir codé
git add .
git commit -m "feat: description de ce que tu as fait"
git push origin feature/nom-de-la-fonctionnalite

# Puis faire une Pull Request sur GitHub
```

 Équipe

Nom	Rôle	GitHub
Regis	Lead Developer	@bossombra1
...	...	...

 Support

Pour toute question sur le projet, contactez le lead developer sur GitHub.